Where TCSlib does not reach: near misses and total misses in the STOC 2026 challenge corpus

lean_formalization pipeline notes

5 November 2026

1 Setup and headline numbers

The corpus is the 259 Lean challenge files generated from the `tcs_v3` STOC 2026 parquets (199 proof-writing, 60 proof-strategy nodes). Each file is a self-contained statement (definitions plus `sorry`-stubbed theorems) that compiles against the pinned TCSlib checkout (Lean v4.25.0, Mathlib `029db123ddaa`). A keyword scan over twenty TCSlib concept areas flagged files whose local definitions overlap a TCSlib definition; two passes of headless refactor agents (66 agents, $25) then rewrote every file where a TCSlib definition could replace a local one without changing the statement. After that pass the corpus splits three ways:

Group	Files	Share	Meaning
Uses TCSlib	61	24%	imports a TCSlib module and states at least one theorem through its definitions
Near miss	37	14%	defines an object TCSlib has, in a shape the paper cannot use
Total miss	161	62%	no local definition corresponds to anything in TCSlib
	259	100%	

The scan originally flagged 61 near-miss files. Nine of them were plain renamings and were converted in the second refactor pass; fifteen were keyword false positives ("walk", "game", "Gaussian", "restriction" used in another sense) and are counted with the total misses here. Earlier notes quoted 68 files (26%) as using TCSlib; that figure double-counted seven files whose agent declined to change anything. The count by actual imports is 61.

2 Near misses: 37 files, fourteen encoding gaps

A near miss is a file that reinvents a TCSlib object because the TCSlib version is indexed, bundled or typed differently than the paper needs. Each gap below names the smallest TCSlib change that would let the files use the library definition. Counts are entries in the gap report; a few files appear under two gaps.

	Gap	Files	Why TCSlib's version does not fit	Fix
C1	Codes	5	`Codeword n` α is `Fin n` $\to \alpha$ and the section needs `[Field` α`]`; papers index words by cube points or an arbitrary finite type	metric definitions over $\iota \to \alpha$ with `[Fintype` ι`]`, plus `relDist`
C2	CNF over generic variables; proof systems	2	`Literal n` is `Fin n`-indexed, clauses are lists; no resolution or Frege	`Literal (V : Type)`, finset clauses, a resolution derivation with width
C3	Decision trees	2	`DecisionTree n` fixes inputs and leaves to `Bool`	parameters for input alphabet and leaf type; `RandDecisionTree` as a `FinDist`
C4	Circuits	7	`AlgCircuit` has only the constant `one`; `DAGCircuit` has no constant wires and carries proof fields; no quantum circuits	`AlgGate.const`, a constant gate kind, an index-type parameter; quantum is a new area
C5	Multigraphs	4	Karger's `Multigraph` is loopless with a multiset of edges	index-based multigraph `ends : Fin m` $\to$ `Sym2 V` with degree, volume, cut edges, conductance
C6	Matchings	4	`Core.Matchings` lives on `SimpleGraph.Subgraph`	`IsMatching` on edge finsets, hyperedges and fractional matchings
C7	Spanning trees and cut edge sets	5	cuts are capacities on a `Network`; no spanning-tree notion	`cutEdges G S` and one `IsSpanningTreeOf` on `Finset (Sym2 V)`
C8	Property testers	6	BLR and low-degree tests exist, but no tester type	`Tester X Y` as a distribution over query trees, `acceptProb`, influence on product spaces
C9	Expanders	2	expansion is spectral only	conductance, Cheeger, existence of explicit expanders
C10	Protocols	2	two-party deterministic, public- and private-coin only	k-party number-in-hand protocols; biased-coin messages
C11	Hermite functions; Reed–Solomon	2	probabilists' Hermite only; no Reed–Solomon or Reed–Muller codes	physicists' family with the rescaling lemma; both code families
C12	Total variation	2	`statDist` is finite only	`tvDist` on `Measure` with the finite case as a lemma
C13	Computed distributions	5	`FinDist` bundles its proofs, so a definition cannot build one	constructors for conditioning, kernel powers, k-fold products, uniform on a subset
C14	Learning	2	external regret for finite experts, minimax, mistake bounds only	Φ-regret; a PAC learner over measures

Ranked by files unlocked per definition added, the order to work in is: index-generic codes (C1, 5 files, a few lines); spanning trees and cut edge sets (C7, 5); edge-finset and hypergraph matchings (C6, 4); an index-based multigraph with loops (C5, 4, which also subsumes Karger's); constant gates in both circuit models (C4 classical half, 4); `FinDist` constructors (C13, 5 files already half

converted); a tester type (C8, 6). Together these fourteen items would move about 35 of the 37 near-miss files into the "uses TCSlib" group. Each is a generalisation of an existing definition, not new mathematics, and most are under a page of Lean.

3 Total misses: 161 files, five areas

A total miss defines nothing TCSlib defines. Grouping the 161 files by the paper's research area (assigned by hand from the title, so the boundaries are approximate):

Area	Files	Papers	What a library would need
Continuous optimisation, convex geometry, lattices, numerical linear algebra, algebra and number theory	50	21	nothing TCS-specific: these statements live on Mathlib's analysis, linear algebra and lattice theory, and TCSlib is the wrong target
Learning, statistics, sampling, Markov chains, hashing, property testing, hardness of approximation	46	24	PAC and agnostic learners over measures, sample complexity, mixing times, measure-level total variation, hash families, CSP and PCP vocabulary
Graph algorithms: shortest paths, distance oracles, planar and minor-free graphs, treewidth, dynamic and streaming algorithms	41	23	weighted shortest-path distances and distance oracles, tree decompositions, planarity, a streaming and dynamic-algorithm model
Proof complexity, circuit lower bounds, pseudorandomness, cryptography, computability	14	10	resolution and Frege systems, algebraic complexity, explicit constructions, a Turing-machine model
Quantum	10	5	states, unitaries, Pauli and Clifford groups, measurements, tomography
	161	83	

Two observations separate this group from the near misses.

- **New areas, not adjusted definitions.** A near miss closes with a one-line generalisation. A total miss needs a vocabulary that does not exist in TCSlib, and for the 50 optimisation and geometry files the right home is Mathlib rather than TCSlib.
- **Most content is paper-specific regardless.** Files average 14 to 16 local definitions in every group (13.6 among total misses, 16.0 among near misses, 16.5 among files that already use TCSlib). A library absorbs the shared vocabulary (a mixing time, a tree decomposition, a PAC learner) but not the per-paper potential functions, gadgets and reductions that make up the bulk of each statement. Even with the five areas above added, these files would look like the current 24%: a handful of library names at the top and a page of local definitions below.

If TCSlib were to grow toward this corpus, the order by files unlocked is a learning and sampling area (46 files), a graph-algorithms area on weighted graphs (41), then quantum (10 here plus 5 near-miss files, the largest single gap by file count but a new library rather than a fix). The optimisation and geometry block should be left to Mathlib.

4 Method and caveats

Membership was computed on 5 November 2026 from the files in `challenges/stoc2026_pw` and `challenges/stoc2026_ps`: a file "uses TCSlib" if it has an `import TCSlib.*` line; the near-miss set is the concept scan's 61 flagged files minus the nine converted and the fifteen false positives; everything else is a total miss. The gap categories C1–C14 come from reading each near-miss file's definitions against the pinned TCSlib source and are recorded with per-file detail in `notes/tcslib_gap_report.md`. The area assignment of total misses is by paper title and would shift a few files under a different rubric (for example, hardness-of-approximation papers sit with learning and statistics here, and lattice-based quantum lower bounds sit with lattices). The per-file refactor notes are in `batches/refactor_tcslib/refactor_notes_all.jsonl` and are reproduced in each file's provenance section in the public fork (`https://github.com/qedsphere/tcslib`, branch `challenges/stoc2026`).